```
console.info("JS loaded");


const Game = (() => {
const yard = document.querySelector("#yard");
const bailee = document.querySelector("#bailee");
const sparkleTrail = document.querySelector("#sparkle-trail");
const hintBubble = document.querySelector("#hint-bubble");
const readToggle = document.querySelector("#read-toggle");
const textScale = document.querySelector("#text-scale");
const reducedMotion = document.querySelector("#reduced-motion");
const highlightToggle = document.querySelector("#highlight-toggle");


const state = {
scene: null,
items: {},
actor: {
position: { x: 0, y: 0 }
},
interactables: new Map(),
hintTimeout: null,
lastMoveTimestamp: 0
};


const fallbackSceneElement = document.querySelector("#fallback-scene");


const setActorPosition = () => {
bailee.style.left = `${state.actor.position.x}%`;
bailee.style.top = `${state.actor.position.y}%`;
};


const fallbackDialogue = {
opening: {
start: "porch-intro",
nodes: {
"porch-intro": {
speaker: "Narrator",
portrait: "narrator",
text: "The porch boards are warm under your sneakers. The backyard smells like sun and grass.",
next: "bailee-snack"
```

```
},
"bailee-snack": {
speaker: "Bailee",
portrait: "bailee",
text: "Mission: backyard adventure. But first—snack power!",
next: null
}
}
}
};


const fallbackScene = {
sceneId: "backyard",
name: "Bailee's Backyard",
startPosition: { x: 20, y: 70 },
interactables: [],
triggers: []
};


const loadData = async () => {
console.info("Loading game data...");
try {
const [dialogueData, sceneData, itemData] = await Promise.all([
fetch("./data/dialogue.json").then((res) => (res.ok ? res.json() : null)),
fetch("./data/scenes.json").then((res) => (res.ok ? res.json() : null)),
fetch("./data/items.json").then((res) => (res.ok ? res.json() : null))
]);
Dialogue.load(dialogueData || fallbackDialogue);
state.scene = sceneData || fallbackScene;
state.items = itemData || {};
console.info("Data loaded");
} catch (error) {
console.warn("Failed to load game data, using fallback.", error);
Dialogue.load(fallbackDialogue);
state.scene = fallbackScene;
state.items = {};
console.info("Data loaded");
}
};


const hideFallbackScene = () => {
```

```
if (fallbackSceneElement) {
fallbackSceneElement.classList.add("hidden");
}
};


const showHint = (text) => {
hintBubble.textContent = text;
hintBubble.classList.add("visible");
};


const hideHint = () => {
hintBubble.classList.remove("visible");
};


const resetHintTimer = () => {
clearTimeout(state.hintTimeout);
hideHint();
state.hintTimeout = setTimeout(() => {
showHint("Try tapping something sparkly near the shed!");
}, 6000);
};


const startDialogue = (dialogueId, onComplete) => {
Dialogue.start(dialogueId, () => {
onComplete?.();
resetHintTimer();
});
};


const handleInteractable = (interactable) => {
if (Dialogue.isActive) {
return;
}
if (interactable.onInteract?.requiresAllSparks && !Progress.hasAllSparks()) {
startDialogue("shed-lock");
return;
}
if (interactable.onInteract?.dialogue) {
startDialogue(interactable.onInteract.dialogue, () => {
```

```
if (interactable.onInteract.item) {
const item = state.items[interactable.onInteract.item];
if (item) {
Inventory.addItem({ id: interactable.onInteract.item, ...item });
}
}
if (interactable.onInteract.sparkleTrail) {
sparkleTrail.classList.add("active");
}
});
}
};

const bindInteractables = () => {
const elements = Array.from(document.querySelectorAll(".interactable"));
elements.forEach((element) => {
const config = state.scene.interactables.find(
(item) => item.id === element.dataset.id
);
if (config) {
state.interactables.set(config.id, { element, config });
}
element.addEventListener("click", () => {
handleInteractable(config);
});

let pressTimer = null;
const startPress = () => {
pressTimer = setTimeout(() => {
showHint(`${config?.name || "Something"} looks interesting.`);
}, 700);
};
const cancelPress = () => clearTimeout(pressTimer);

element.addEventListener("touchstart", startPress);
element.addEventListener("touchend", cancelPress);
element.addEventListener("touchmove", cancelPress);
element.addEventListener("contextmenu", (event) => {
event.preventDefault();
showHint(`${config?.name || "Something"} might have a secret.`);
});
```

```
});
};


const handleTriggers = () => {
if (!state.scene?.triggers) {
return;
}
state.scene.triggers.forEach((trigger) => {
if (trigger.fired) {
return;
}
const target = state.interactables.get(trigger.target);
if (!target) {
return;
}
const targetRect = target.element.getBoundingClientRect();
const yardRect = yard.getBoundingClientRect();
const targetX = ((targetRect.left + targetRect.width / 2 - yardRect.left) / yardRect.width)
* 100;
const targetY = ((targetRect.top + targetRect.height / 2 - yardRect.top) / yardRect.height)
* 100;
const distance = Math.hypot(targetX - state.actor.position.x, targetY -
state.actor.position.y);
if (distance < trigger.range) {
trigger.fired = true;
startDialogue(trigger.dialogue, () => {
if (trigger.spark) {
Progress.addSpark(trigger.spark);
}
});
}
});
};


const updateShimmer = () => {
state.interactables.forEach(({ element, config }) => {
const rect = element.getBoundingClientRect();
const yardRect = yard.getBoundingClientRect();
const objX = ((rect.left + rect.width / 2 - yardRect.left) / yardRect.width) * 100;
const objY = ((rect.top + rect.height / 2 - yardRect.top) / yardRect.height) * 100;
const distance = Math.hypot(objX - state.actor.position.x, objY - state.actor.position.y);
const shouldShimmer = highlightToggle.checked && distance < config.range;
```

```
    element.classList.toggle("shimmer", shouldShimmer);
  });
};

const handleMove = (event) => {
  if (Dialogue.isActive) {
    return;
  }
  const rect = yard.getBoundingClientRect();
  const x = ((event.clientX - rect.left) / rect.width) * 100;
  const y = ((event.clientY - rect.top) / rect.height) * 100;
  Movement.setTarget(x, y);
  resetHintTimer();
};

const gameLoop = (timestamp) => {
  const delta = timestamp - state.lastMoveTimestamp;
  state.lastMoveTimestamp = timestamp;
  if (!Dialogue.isActive) {
    Movement.update(delta, state.actor);
  }
  bailee.classList.toggle("idle", !Movement.isMoving);
  setActorPosition();
  updateShimmer();
  handleTriggers();
  requestAnimationFrame(gameLoop);
};

const bindUi = () => {
  yard.addEventListener("click", handleMove);
  readToggle.addEventListener("click", () => {
    const pressed = readToggle.getAttribute("aria-pressed") === "true";
    readToggle.setAttribute("aria-pressed", String(!pressed));
    readToggle.textContent = pressed ? "🔊 Read" : "🔊 Reading";
  });

  textScale.addEventListener("input", (event) => {
    document.documentElement.style.fontSize = `${event.target.value}rem`;
  });
```

```

reducedMotion.addEventListener("change", (event) => {
document.body.classList.toggle("reduced-motion", event.target.checked);
});
};

const start = () => {
console.info("Starting The Shed preview...");
Dialogue.load(fallbackDialogue);
state.scene = fallbackScene;
state.items = {};
Inventory.render();
bindUi();
bindInteractables();
hideFallbackScene();
if (state.scene?.startPosition) {
state.actor.position = { ...state.scene.startPosition };
} else {
state.actor.position = { x: 20, y: 70 };
}
setActorPosition();
Progress.update();
resetHintTimer();
if (Dialogue?.start) {
startDialogue("opening");
} else {
console.warn("Dialogue system unavailable; skipping intro.");
}
requestAnimationFrame(gameLoop);

loadData().then(() => {
bindInteractables();
});
};

return { start };
})();

try {
```

```
Game.start();
} catch (error) {
console.error("Game failed to start.", error);
}
```